CSI142 Object-Oriented Programming

Methods and Modularity I

Week 3 • Lecture 7

methods • parameters • return values • pass-by-valueScope and lifetime refresher

Fri 9 Feb 2026 • 08:00–09:00 • Room 252-008

Where we are in the course

- Week 1–2: control flow, arrays/ArrayList, input, command-line arguments
- Week 3: methods and modularity (build reusable building blocks)
- Week 4: classes and objects (methods become behaviours of objects)
- Goal today: write small methods with clear inputs, clear outputs, and minimal side effects

Learning outcomes (today)

- Explain why methods improve readability, reuse, and testing
- Write method signatures with appropriate *parameters* and *return types*
- Trace method calls and predict outputs (including *parameter passing*)
- Apply Java's pass-by-value rule to primitives and references (arrays, objects)
- Use local scope intentionally to prevent bugs

Why methods?

Before: one long main()

- Repeated code blocks
- Hard to debug (everything is connected)
- Hard to test one piece
- One change causes many edits

After: small focused methods

- Each method does one job
- Reuse by calling the method
- Test methods in isolation
- Code reads like a story

Method anatomy (signature)

```
public static int add(int a, int b)
{
    return a + b;
}
```

- access modifier: public (we will use public for now)
- static: belongs to the class (until we reach objects in Week 4)
- return type: int (or void if nothing is returned)
- name: add (use meaningful verbs)
- parameters: (int a, int b) are inputs
- body: the statements inside { }

Example: a simple returning method

```
1 static int square(int x) {
2     return x * x;
3 }
4
5 public static void main(String[]
args) {
6     int n = 6;
7     int result = square(n);
8     System.out.println(result);
9 }
```

Key points

- Call the method by name, pass arguments
- Return value can be stored or printed
- Method works for any x

void vs returning a value

void method

- Performs an action (print, save, update)
- No value to assign
- Call it as a statement:

```
printMenu();
```

returning method

- Computes an answer
- Value can be used later
- Call it inside an expression:

```
avg = computeAverage(...);
```

Parameter passing in Java (the rule)

Primitives (int, double, char, boolean)

- Argument value is copied into the parameter
- Changing the parameter does not change the caller variable
- Example: increment(x) does not modify x

References (arrays, objects)

- The reference value is copied (address-like value)
- Both variables refer to the same object
- Changing the object can be seen by the caller

Pass-by-value: primitives vs arrays

```
1 static void increment(int n) {
2     n = n + 1;
3 }
4
5 static void addBonus(int[] marks) {
6     marks[0] = marks[0] + 5;
7 }
8
9 public static void main(String[] args) {
10     int x = 10;
11     int[] a = {50, 60};
12     increment(x);
13     addBonus(a);
14     System.out.println(x);
15     System.out.println(a[0]);
16 }
```

Key points

- x stays 10 (primitive copy)
- a[0] changes (same array object)
- Predict first, then run

Scope: where variables exist

```
1 public static void main(String[]
args) {
2       int x = 1;
3       if (x == 1) {
4           int y = 2;
5           System.out.println(x +
y);
6       }
7       // System.out.println(y);  //
compile error
8 }
```

Key points

- Variables live inside { }
- Parameters are local to a method
- Small scope reduces bugs

Example: refactoring a loop into a method

```
1 static int calculateSum(int[] values) {
2     int total = 0;
3     for (int v : values) {
4         total += v;
5     }
6     return total;
7 }
8
9 public static void main(String[] args) {
10     int[] scores = {10, 20, 30};
11
System.out.println(calculateSum(scores));
12 }
```

Key points

- Method hides the loop details
- Enhanced for-loop is readable
- Return enables reuse

Activity: plan methods before coding

Pair task (3 minutes):
Given a program that reads non-negative integers until -1, then prints the average:

1) Propose 3 method names you would create
2) Write the method signatures (parameters and return types)
3) Identify what should stay in main()

Common pitfalls to avoid

1. Method does too many things, split it
2. ***Unclear names:*** prefer verbs like compute, validate, format
3. ***Too many parameters:*** consider grouping later (preview: objects)
4. ***Returning and printing at the same time:*** choose one responsibility
5. ***Forgetting to test edge cases:*** empty input, divide by zero

Wrap-up and next steps

- Methods turn repeated logic into reusable building blocks
- Java passes everything by value (including object references)
- Scope is your friend, keep variables local
- Next lecture: method overloading and scope in more detail